P-192 three-coordinate marked orbit representatives

Three cached points -> affine lattice relation

Index r; four exact coefficient minima

Construct missing points -> canonical representative

Triple MSM, sum-zero identity, explicit scalar label

Independent replay

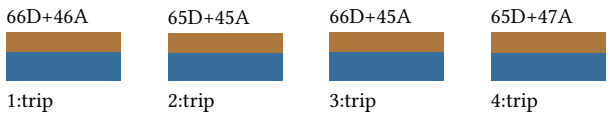
100 scalar images and 160 equivalent encoding controls

Store R, lambda R, lambda^2 R for each nonidentity fifth root lambda = mu^j modulo the prime subgroup order r. Solve c0 + lambda c1 + lambda^2 c2 = lambda^3 modulo r. A three-input signed-digit multiplication constructs the fourth point; the sum-zero cyclotomic identity constructs the fifth. Minimum x, then minimum signed y, selects an orbit representative. Retain three consecutive points and multiply scalar labels by sign * lambda^rotation.

The affine lattice has index r. Exact inverse-image enumeration proves all four global minimum coefficient heights; each is 63 bits. This proves a coefficient minimum, not an arithmetic-circuit minimum. The schedule search is separately restricted to absolute coefficients at most 2^65-1 and widths 2 through 6. It contains 250 relations and 31,250 schedules. Larger-than-minimum coefficients can have cheaper signed-digit schedules.

Checked representative costs. Powers 1, 2, 3, 4 respectively require 66D+46A, 65D+45A, 66D+45A, 65D+47A. D and A are group doublings and additions. These totals include all three odd-multiple tables and the three additions deriving the fifth point. Negated lookups, final sign changes and comparisons are recorded separately. Initial computation of the stored triple is excluded and must be charged when used in an attack.

Round46: complete canonical representative, group operations



Blue: doublings. Brown: additions. Three input tables included.

Exact schedules, not timing; input-triple setup, negations and comparisons separate.

Replay. verify.py independently enumerates an alternate unimodular basis using exact rational inversion, checks the empty cube below each minimum, and recomputes every schedule. points.py evaluates the selected schedules on five public fixtures per eigenvalue, including identity. verify_points.py imports no producer arithmetic: it checks 100 direct scalar images, 160 rotated/signed encoding controls, and the representative scalar labels. All checks pass. These are correctness fixtures, not random-walk measurements.

Cache tradeoff. The preceding two-coordinate construction has a best checked schedule of 187D+80A. A three-coordinate state uses 144 raw point bytes plus 48 scalar-label bytes, compared with 96 bytes for an ordinary labelled point. Four cached points allow construction of the fifth using three additions; five cached points need no missing-point reconstruction. Translation requires respectively 2, 3, 4 or 5 coordinate additions before canonicalization. These are different storage and setup costs; the D+A schedule objective does not establish a field-operation or elapsed-time improvement.

The explicit point action supplies the scalar-label equation that curve-identity collisions alone lack. A collision of canonical labelled states (a,b) and (c,d) gives x=(a-c)/(d-b) modulo r only when d-b is nonzero. The next construction is a full labelled three-coordinate transition with counted field arithmetic, setup and representative costs, plus cycle controls. The present result supplies its representative subroutine.